Query Tuning in MySQL

- Overview of optimization techniques
- Why Query Tuning? - Improves performance and reduces load
- Indexes speed up data retrieval
- Avoid SELECT * - Fetch only required columns
- Fetch only required columns - Analyze query execution plan

contd..

- Optimize Joins Use proper join types and indexes
- Limit Results - Use LIMIT to reduce dataset
- Avoid Subqueries - Use joins instead where possible
- Use Proper Data Types - Efficient storage and comparison
- Conclusion - Better queries = faster applications

Query Tuning in MySQL

- Using real examples: EXPLAIN, Indexes, Joins
- Sample Table –
  - ✓ CREATE TABLE users(id INT, name VARCHAR(50), age INT);
  - ✓ INSERT INTO users VALUES (1,'A',25);
- Slow Query Example - SELECT * FROM users WHERE age = 25;

EXPLAIN Output

- EXPLAIN SELECT * FROM users WHERE age=25;
- Type: ALL (Full Table Scan)
- Add Index - CREATE INDEX idx_age ON users(age);
- Optimized Query - EXPLAIN SELECT * FROM users WHERE age=25;
  Type: ref (Using Index)

EXPLAIN Optimized

- After index:
- CREATE INDEX idx_age ON users(age);
- id: 1
- type: ref (good)
- key: idx_age
- rows: 100

Composite Index

- CREATE INDEX idx_user_age_name
- ON users(age, name);
- Best when:
- WHERE age=25 AND name='A'
- Order matters!

Composite Index Rule

- Works for:
- (age)
- (age, name)

- Not for:
- (name) alone

- Leftmost prefix rule

Composite Index Rule

- Works for:
- (age)
- (age, name)

- Not for:
- (name) alone

- Leftmost prefix rule

contd..

Optimize GROUP BY and HAVING

Filtering data early using WHERE is much faster than filtering after aggregation with HAVING.

- **Best Practice:** Move conditions to the WHERE clause whenever possible to reduce the number of rows processed by OneUptime.
- **Example:**

sql

-- *Inefficient: Aggregates all rows first* SELECT region, SUM(total) FROM sales GROUP BY region HAVING region = 'North'; -- *Optimized: Filters rows before aggregation* SELECT region, SUM(total) FROM sales WHERE region = 'North' GROUP BY region;

Join Example

- SELECT u.name, o.amount FROM users u JOIN orders o ON u.id=o.user_id;
- Bad Join - No index on user_id causes full scan
- Optimized Join –

  a. CREATE INDEX idx_user_id ON orders(user_id);
- Conclusion - Indexes + EXPLAIN = Better Performance

Join Optimization

- SELECT u.name, o.amount
- FROM users u
- JOIN orders o
- ON u.id=o.user_id;

- Index on orders(user_id) required

Before Optimization

- type: ALL
- rows: 10,000,000
- Execution time: High

After Optimization

- type: ref
- rows: 1000
- Execution time: Low

contd..

Limit Result Set – Using the **LIMIT clause** to restrict the number of rows returned especially for the queries fetching large datasets can improve query performance and reduce resource consumption.

Example: *SELECT * FROM products LIMIT 10;*

Avoid Nested Queries – The Rewrite nested queries as **JOIN** operations to improve the query readability and performance. The Nested queries can often be inefficient especially when dealing with large datasets.

Example: *SELECT * FROM employees JOIN departments ON employees.department_id = departments. department_id;*

Utilize Query Caching

- Implementing caching mechanisms to store frequently **accessed data** can significantly reduce query execution overhead and improve performance.
- Query caching can store the results of a query and return the cached result for **subsequent requests**, **reducing** the need to re-execute the same query multiple times.

contd..

- **Example:**
- **-- Example of setting up query cache (MySQL configuration)**
  -- Enable query caching in the MySQL configuration file (my.cnf or my.ini)

  [mysqld]
  query_cache_type = 1
  query_cache_size = 16M
- With query caching enabled, frequently executed queries can be served from the **cache**, leading to faster response times and reduced load on the database server.

Best Practices

- ✓ Use EXPLAIN
- ✓ Add indexes
- ✓ Avoid SELECT *
- ✓ Optimize joins

Best Practices for MySQL Query Optimization

- Regularly analyze slow queries using the profiling tools and optimize them based on the performance metrics.
- Monitor database **performance** metrics to identify optimization opportunities.
- Implement caching mechanisms to store frequently accessed data and reduce query execution overhead.
- Regularly update **MySQL server** and optimizer versions to leverage performance **improvements** and **bug fixes**.
- They partition large tables to distribute data across multiple physical storage devices and improve query performance.
- **Optimize storage** engine configuration parameters based on the workload characteristics and available system resources.

Conclusion

- Query tuning reduces load
- Improves performance drastically

Performance Tuning

- Performance tuning in MySQL involves optimizing hardware, system configuration, and SQL code to ensure faster data retrieval and efficient resource usage.

Key Configuration Parameters

Adjusting global variables in my.cnf or my.ini can provide massive gains.

•**innodb_buffer_pool_size**: The most critical setting for InnoDB. It caches data and indexes in RAM. On dedicated servers, set this to 50–80% of total system memory.

•**innodb_log_file_size**: Sets the size of redo logs. Larger logs can improve write performance by reducing the frequency of background flushes. A good rule is for logs to hold one hour's worth of data.

•**max_connections**: Ensure this is high enough to handle peak traffic but low enough to avoid exhausting RAM.

•**query_cache_size**: Note that the Query Cache was removed in MySQL 8.0. For modern versions, use application-level caching like Redis.

contd..

4. System-Level Tuning

Underlying hardware and OS settings directly impact database speed.

- **Storage**: Use SSDs over HDDs for drastically higher IOPS (Input/Output Operations Per Second).
- **Linux Swappiness**: Set vm.swappiness to a low value (e.g., 1 or 10) to prevent the OS from moving MySQL data out of RAM to slower disk swap space.
- **I/O Scheduler**: For SSDs, using the deadline or noop schedulers often yields better performance than the default.

contd..

5. Essential Tuning Tools

- MySQLTuner: A Perl script that analyzes your current status and provides specific configuration change recommendations.
- Percona Monitoring and Management (PMM): A free, open-source tool for advanced monitoring and deep analysis of query performance.
- pt-query-digest: Part of the Percona Toolkit, this analyzes slow query logs to identify the "heaviest" queries hitting your server.